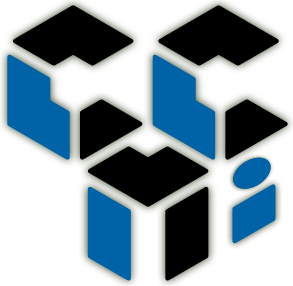
OntoUML: Převod na objektový model



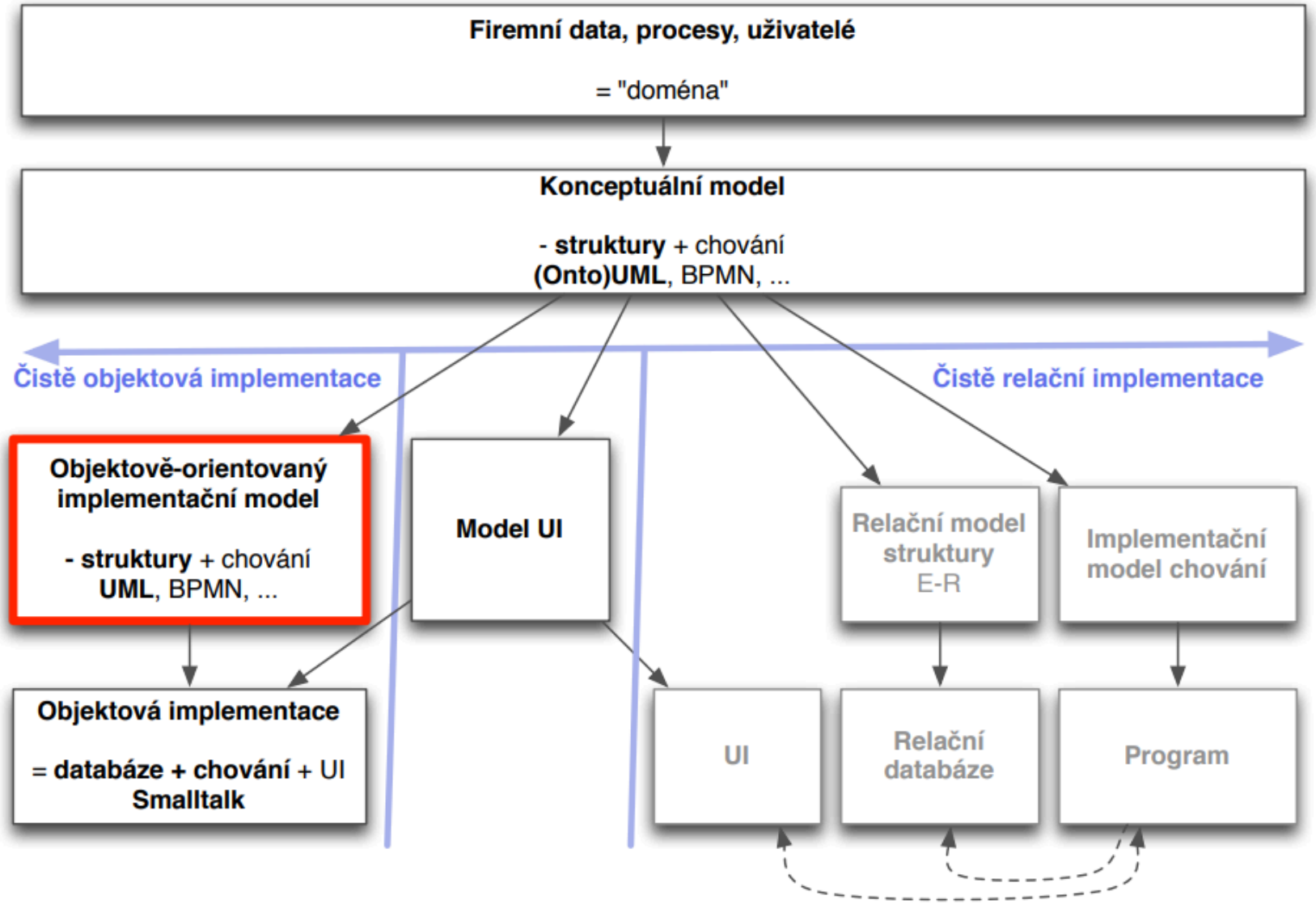


FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

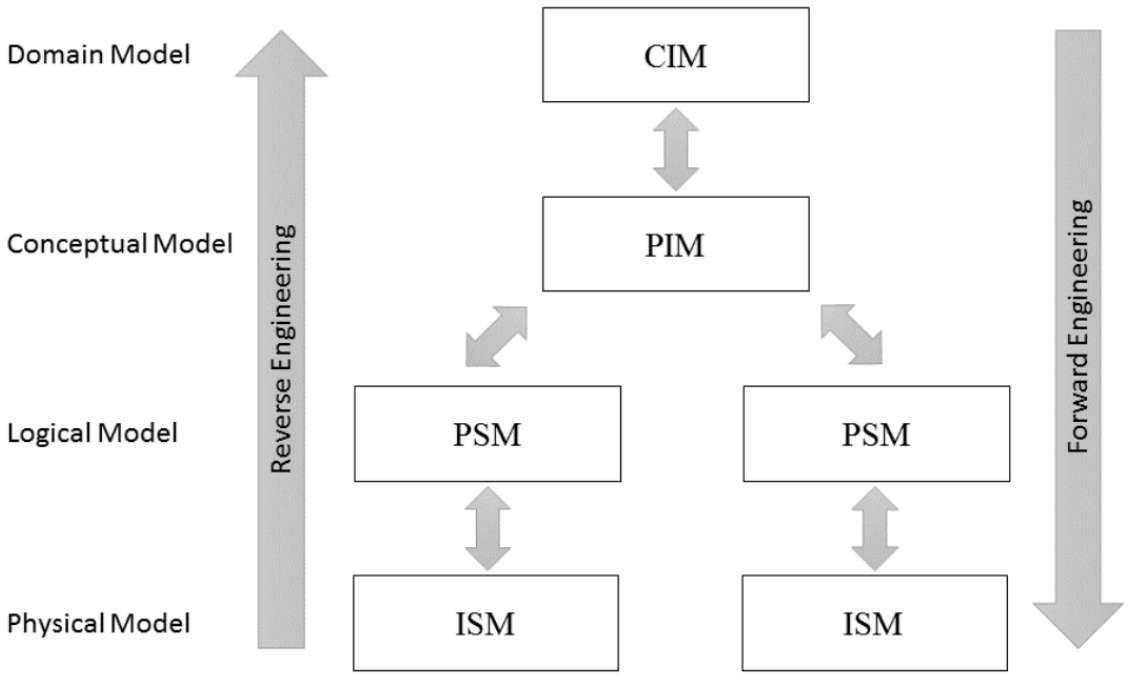
BI-KOM: Konceptuální modelování #6

doc. Ing. Robert Pergl, Ph.D. (robert.pergl@fit.cvut.cz)

Transformace OntoUML na objektový model



Model-Driven Development

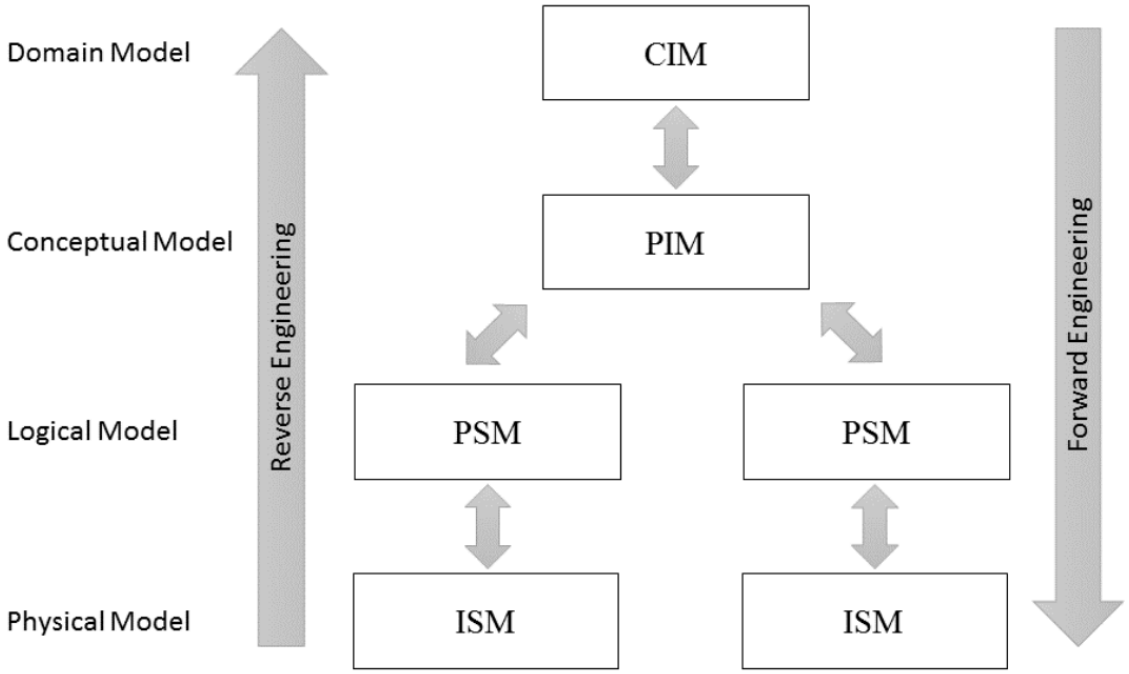


Computation Independent Model (Domain Model)

Describes the environment, business processes and business requirements for the product, abstracted from any implementation & technologies.

Rybola, Z., 2017. Towards OntoUML for Software Engineering: Transformation of OntoUML into Relational Databases (Ph.D.). Czech Technical University in Prague, Czech Republic

Model-Driven Development

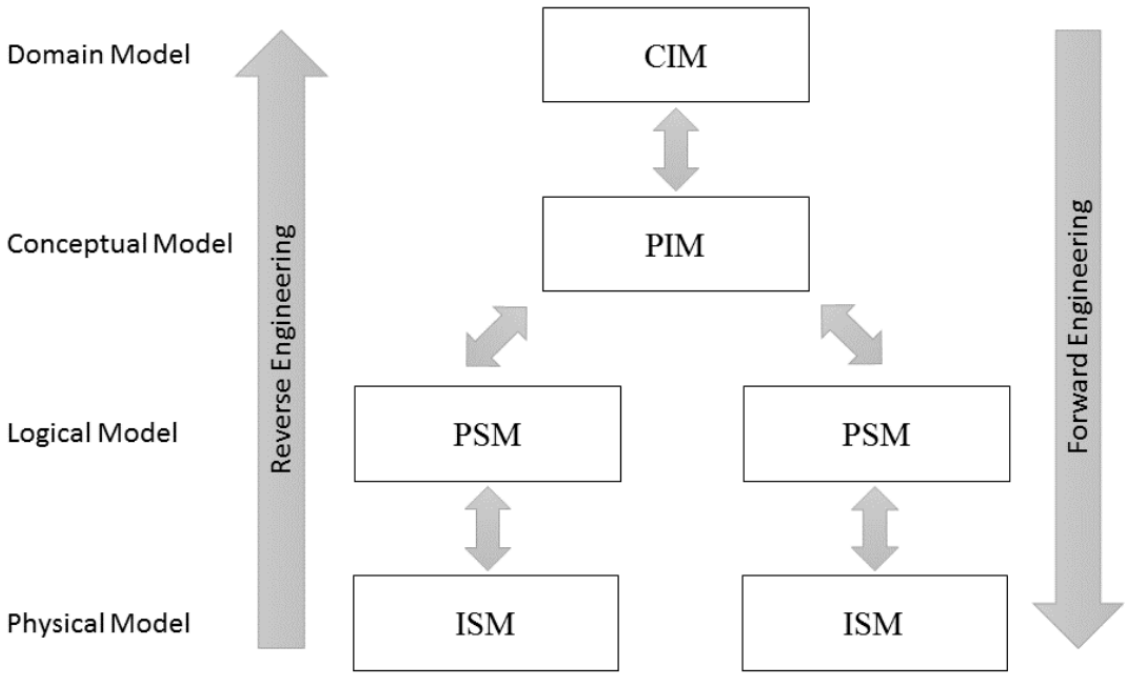


Platform Independent Model (Analytical Model)

User requirements and specifications of the system, but still abstracted from implementation & technologies.

Rybola, Z., 2017. Towards OntoUML for Software Engineering: Transformation of OntoUML into Relational Databases (Ph.D.). Czech Technical University in Prague, Czech Republic

Model-Driven Development

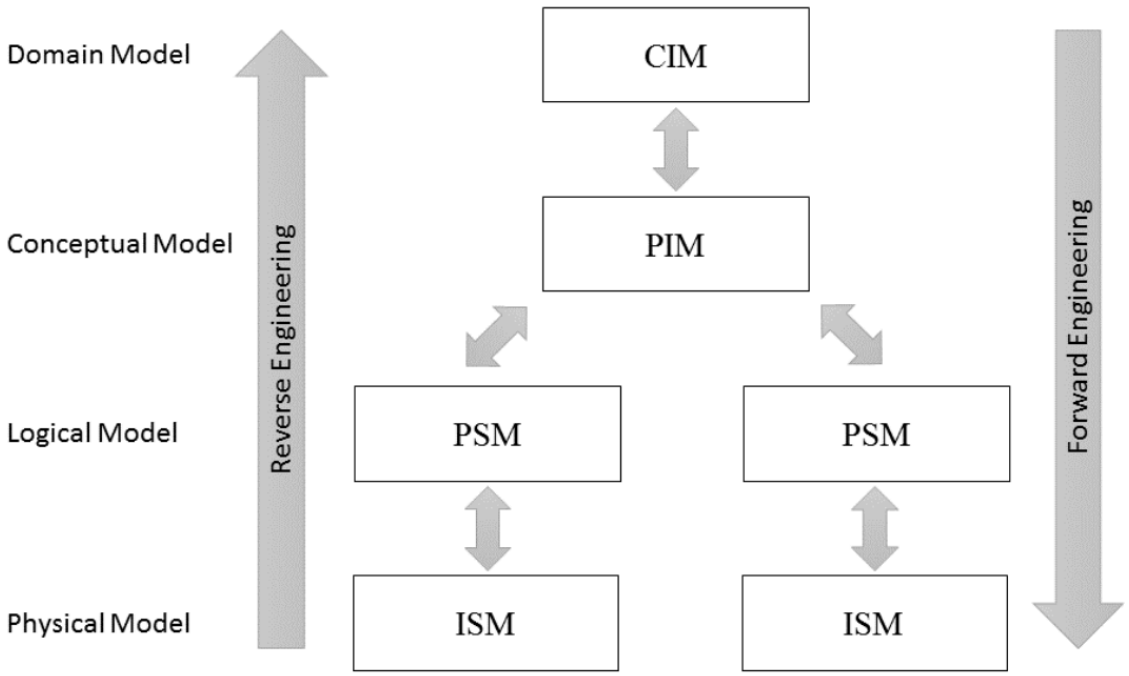


Platform Specific Model (Logical Model)

Describes the design of the system for a specific platform, i.e. how the system requirements defined in the PIM are realized using specific technologies.

Rybola, Z., 2017. Towards OntoUML for Software Engineering: Transformation of OntoUML into Relational Databases (Ph.D.). Czech Technical University in Prague, Czech Republic

Model-Driven Development



Implementation Specific Model (Physical Model)

Actual code of the system with its implementation documentation.

Rybola, Z., 2017. Towards OntoUML for Software Engineering: Transformation of OntoUML into Relational Databases (Ph.D.). Czech Technical University in Prague, Czech Republic

Obecné principy transformace

- Objektový model obsahuje méně konstruktů než OntoUML, OntoUML model tedy bude při transformaci *degenerovat*, a to na tyto OO konstrukty:
  - objekt (třída) s atributy (a metodami),
  - skládání objektů,
  - dědění mezi třídami.
- Principy a vzory transformace jsou do určité míry využitelné i pro transformace do jiných modelů, např. OWL, přednáška 8.
- Transformace do relační DB je již mimo obsah tohoto předmětu. Pro zájemce je k dispozici práce Dr. Ryboly.

Transformace entit

- Různé typy entit OntoUML («Kind», «Role», «Mixin», ...) se transformují na třídy.
- Transformace atributů je 1:1 s tím, že
  - Typy můžeme namapovat již na konkrétní OO jazyk, nebo ponechat na konceptuální úrovni.
  - Multiplicity atributů `0..1` vedou na instanční proměnné.
  - Multiplicity atributů `0..*` vedou na typ kolekce.
  - U multiplicit atributů `1..1` je nutno zajistit neprázdnou hodnotu instanční proměnné.
  - U multiplicit atributů `1..*` je nutno zajistit neprázdnost kolekce.

Zajištění povinnosti atributů (multiplicity 1..)

1. **Hard metoda**: vynucení zapsáním hodnoty při vytváření objektu:

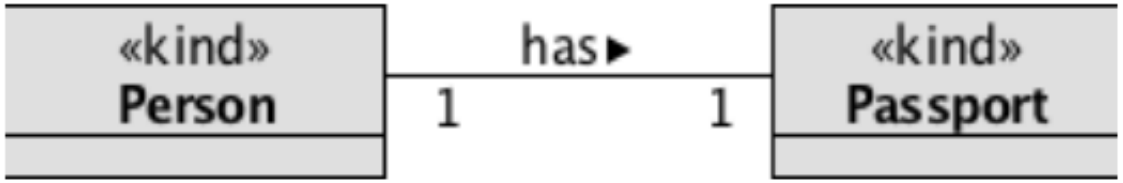
- Použijeme *konstruktor*, tj. hodnotu atributu naplníme již při vytváření objektu.
- Pokud jazyk umožňuje, zabráníme vytváření instance (ve ST předefinování třídní metody `new`).

2. **Soft metoda**: při vytváření objektu neřešíme, ale implementujeme kontroly konzistence jakožto metody, které jsou volány v určitý okamžik: např. necháme uživatele libovolně vyplňovat hodnoty instančních proměnných ve formuláři a konzistenci kontrolujeme před použitím v kódu.

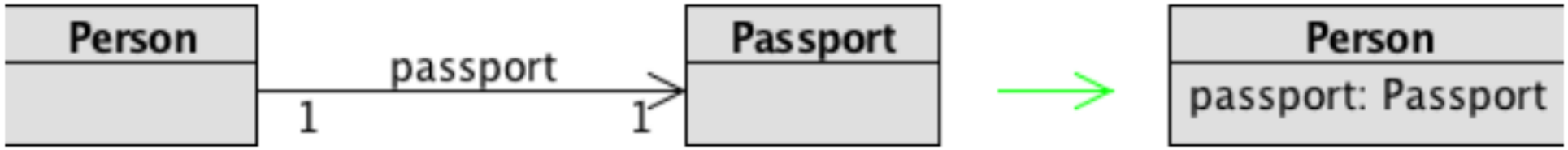
Zajištění asociací

- Asociace je vztah **obousměrný**, který má název a **multiplicity na obou stranách, tj. 4 charakteristiky** (2× spodní mez, 2× horní mez).
- Mezi třídami A a B můžeme rozlišit 3 typy asociací:
  - **1:1** – každá instance třídy A má přiřazenu (maximálně) jednu instanci třídy B.
  - **1:N** – instance třídy A může mít přiřazeno i více instancí třídy B. Každá instance třídy B má však přiřazenu max. jednu instanci třídy A.
  - **M:N** – instance třídy A může mít přiřazeno více instancí třídy B a instance třídy B může mít přiřazeno více instancí třídy A.
- Skládání je však vztah **jednosměrný**...

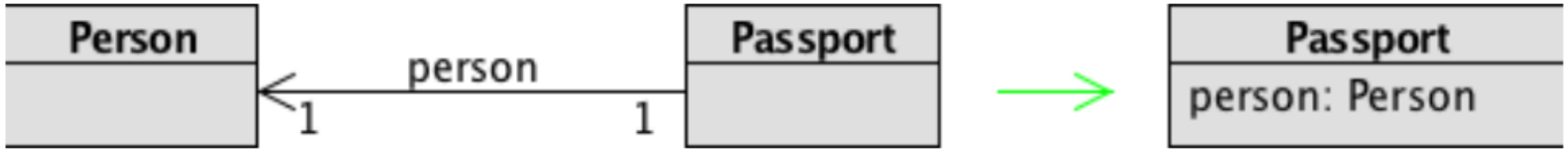
Implementace asociace typu 1:1



- Navigace `Person` → `Passport`

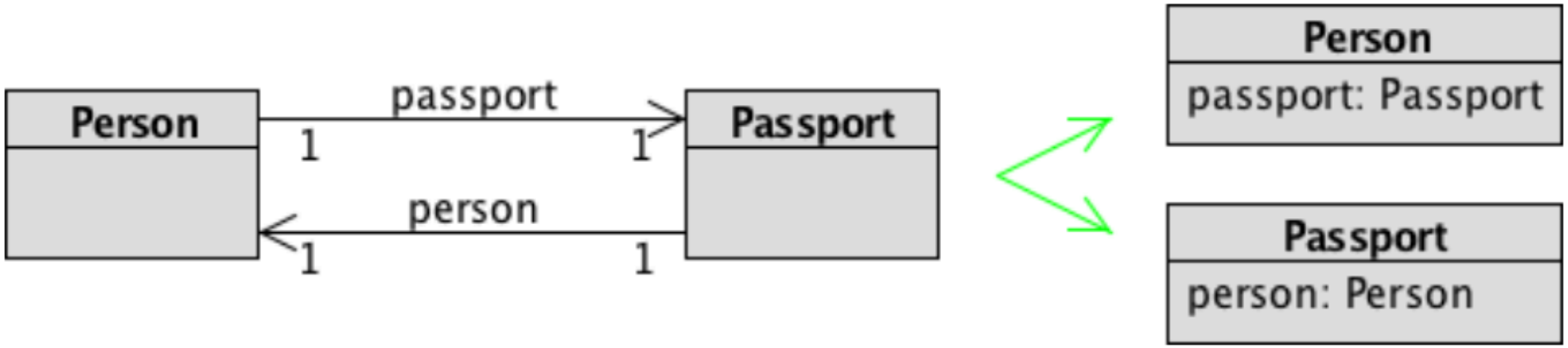


- Navigace `Person` ← `Passport`



Implementace asociace typu 1:1

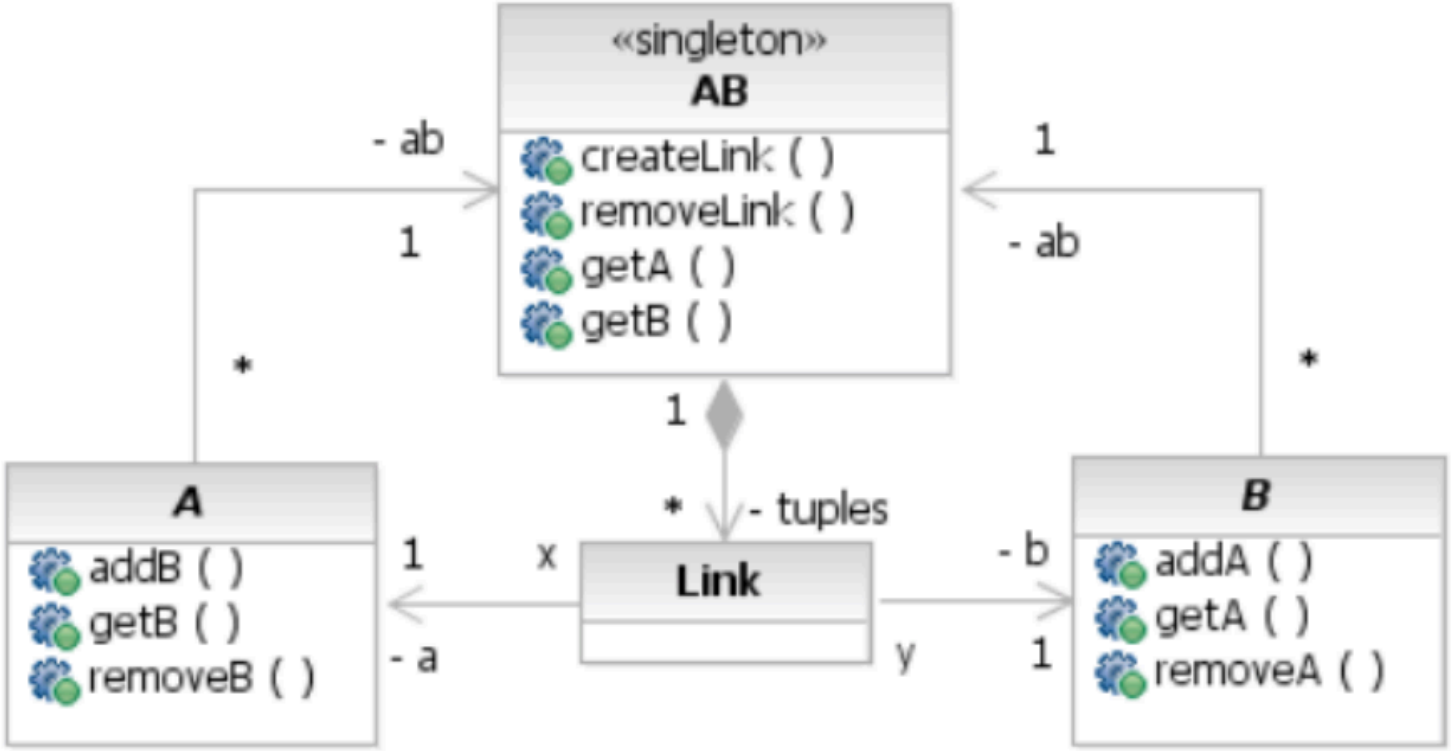
- Obousměrná navigace `Person` ↔ `Passport`



Implementace asociace typu 1:1

- Volba implementace závisí na budoucím použití:
  - Databáze osob, u nichž nás zajímají pasy: `Person` $\longrightarrow$ `Passport`
  - Databáze pasů, k nim nás zajímají osoby: `Person` $\longleftarrow$ `Passport`
- Implementace `Person` $\longleftrightarrow$ `Passport` jsou tzv. *brusle*. Snažíme se jí vyhnout:
- Vkládání a odebírání asociace je třeba ošetřit u obou objektů $\longrightarrow$ **nebezpečí vzniku nekonzistencí**.

Relationship Object Pattern



Dominik Gessenharter. Implementing UML Associations in Java: A Slim Code Pattern for a Complex Modeling Concept. In: Proceedings of the Workshop on Relationships and Associations in Object-Oriented Languages, 17–24. RAOOL '09. New York, NY, 2009.

Implementace asociace typu 1:1

- Tento návrhový vzor řeší riziko nekonzistencí.
- Je použitelný i pro typy vztahů 1:N a M:N (viz dále).
- Nicméně přidává další složitost =>
- Implementujeme až jako **optimalizaci**, kdy potřebujeme provádět efektivně dotazy z obou stran
- Vhodnou volbou jednosměrné navigace usnadníme implementaci dotazů, zajistíme efektivitu a udržovatelnost (KISS Principle).

Ošetření multiplicity na straně šipky

- Je třeba ošetřit spodní mez multiplicity:
  - Jelikož skládání vede na vytváření atributů, aplikujeme **hard** nebo **soft** metodu.
  - Platí i pro typy asociací **1:N** a **M:N**.

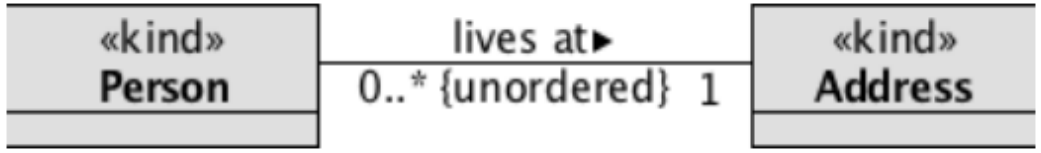


Ošetření multiplicity na straně bez šipky

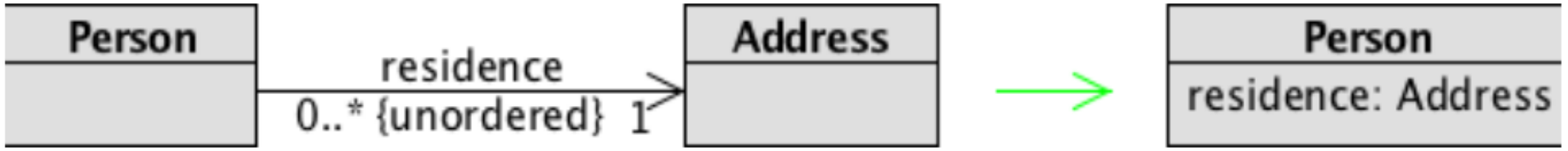
- Určuje, kolik instancí celku může skládat jednu instanci části. Je třeba programově kontrolovat, aby:
  - **0..1** – instance byla obsažena max. v jednom celku,
  - **1** – instance byla obsažena právě v jednom celku.



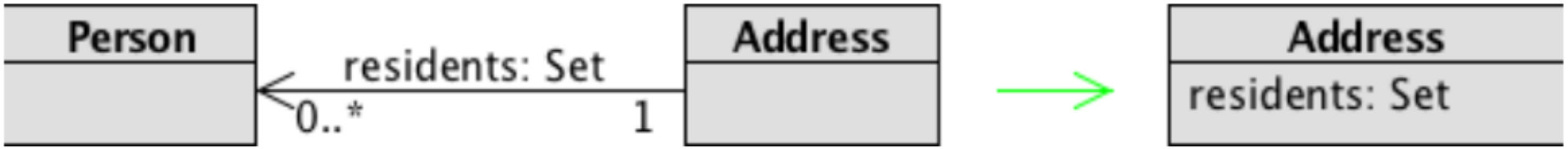
Implementace asociace typu 1:N



- Navigace `Person` → `Address`

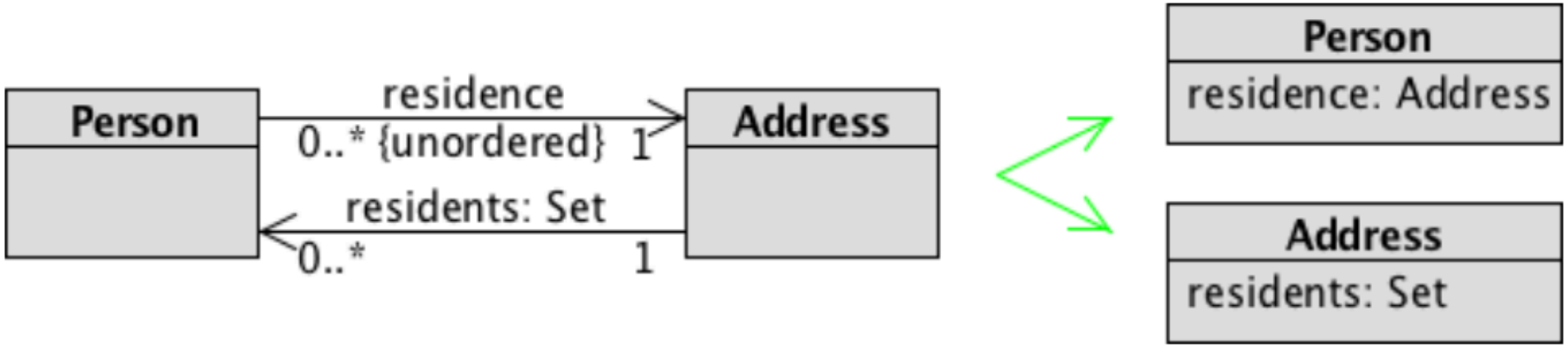


- Navigace `Person` ← `Address`



Implementace asociace typu 1:N

- Obousměrná navigace `Person` ↔ `Address`

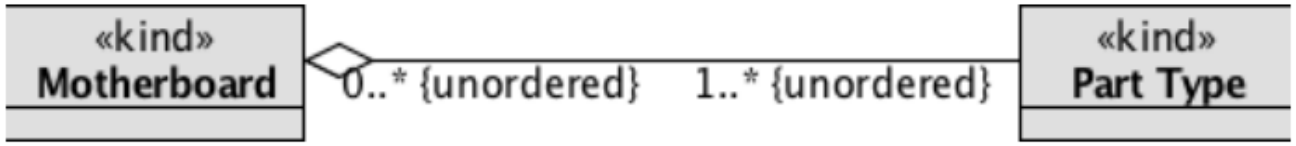


Ošetření multiplicity na straně bez šipky

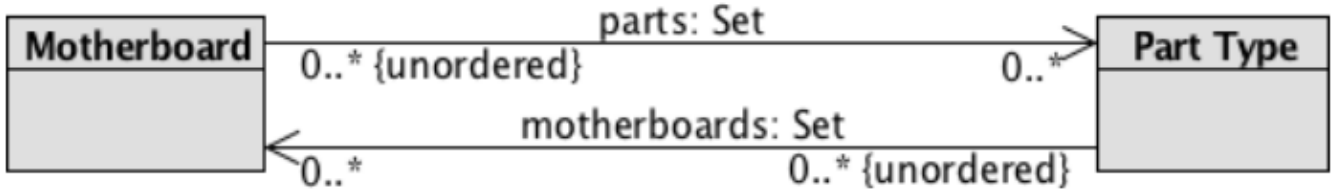
- Určuje, kolik instancí celku může skládat jednu instanci části. Je třeba programově kontrolovat, aby:
  - **0..1** – instance byla obsažena max. v jednom celku,
  - **1** – instance byla obsažena právě v jednom celku,
  - **0..n** – libovolné množství celků, které skládají část = žádná kontrola,
  - **1..n** – prvek musí být obsažen alespoň v jedné kolekci.

Implementace asociace typu M:N

- Jak bylo vysvětleno v přednášce o Relacích, materiální vztahy M:N se použitím «Relator» rozloží na dva vztahy 1:N.
- S potřebou implementovat vztah typu M:N se však setkáme v případně vztahů Celek-Část:



- Na rozdíl od relačního modelu lze v objektovém modelu vazbu implementovat přímo, bez vazebního objektu: (viz též Relationship Object Pattern a diskuse)



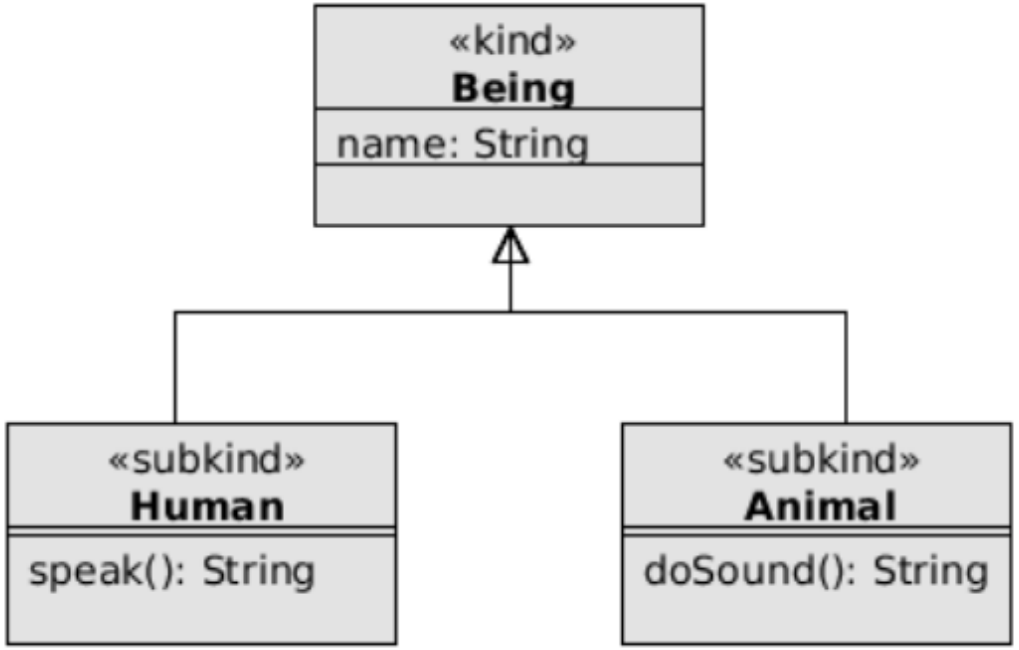
Implementace metaatributu dědění `Complete`

- Metaatribut `complete` znamená, že neexistují žádné instance, které by byly výhradně instancí nadtřídy (a tedy žádné z podtříd).
- V OO implementujeme jako *abstraktní třídu*.
- Naopak, pokud metaatribut `complete` není přítomen, třída není abstraktní.

Implementace metaatributu dědění `Disjoint`

- Metaatribut `disjoint` znamená, že instance třídy bude instancí nejvíce jedné z podtříd.
- Toto je standardní chování objektového systému.
- Ovšem v případě, že metaatribut `disjoint` není přítomen (je *non-disjoint*), potřebujeme zavést instanci, jež je instancí vícero tříd.
- Toto nelze v OO systému obvykle přímo realizovat.

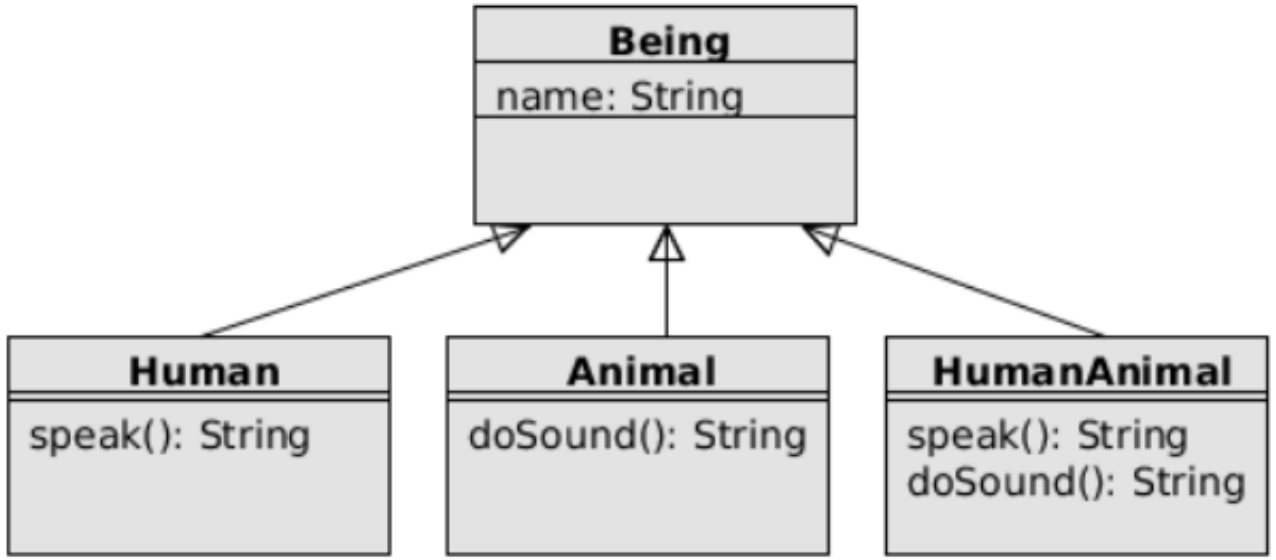
Implementace non-disjoint dědění



- Instance mohou být libovolnou kombinací `Being`, `Human` a `Animal`.
- Možnosti realizace v OO:
  i. **dědění** (inheritance)
  ii. **skládání** (composition)

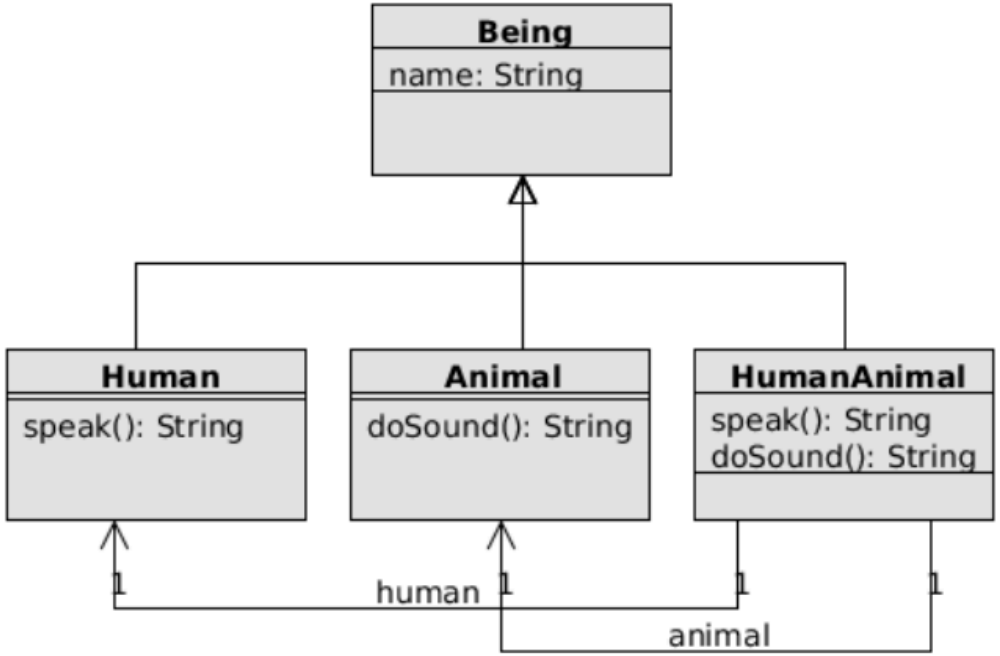
Implementace non-disjoint pomocí dědění

- Každá podtřída bude reprezentovat jednu možnou kombinaci = vidíme typy instancí.
- Musíme tedy vytvořit tolik podtříd, kolik je možných kombinací:



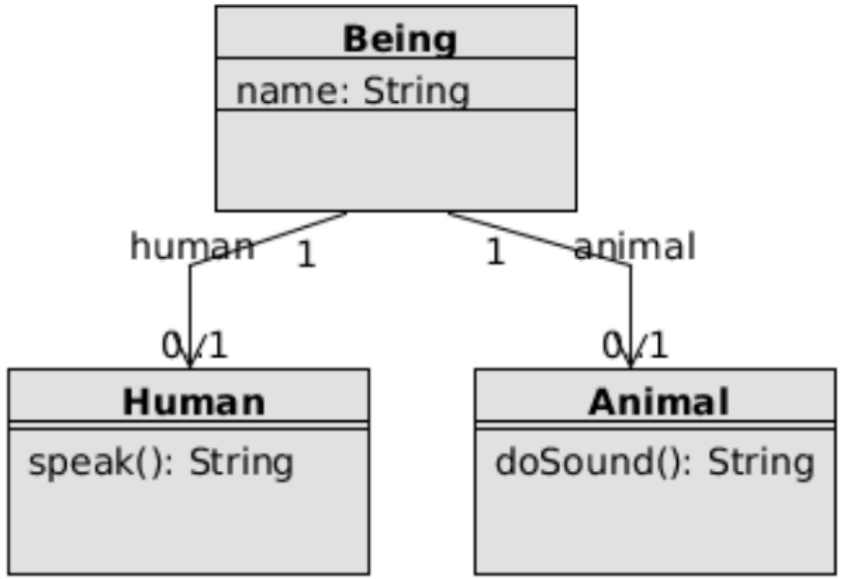
- Počet kombinací ale roste exponenciálně (závažné porušení **DRY**):
  - Používat jen u malého počtu tříd.
  - Nevytvářet kombinace, které víme, že nemohou nastat.

Implementace non-disjoint pomocí dědění a skládání



- Využit princip **delegování**.
- Neporušuje DRY.
- Stále obecně nutný exponenciální počet podtříd.

Implementace non-disjoint pomocí čistého skládání



- Počet instancí roste lineárně.
- Ztrácíme přehled o vazbě dědění.
- Nelze použít pro `complete` (nadtřída nemůže být abstract).

Alternativní implementace

- Některé OO jazyky obsahují speciální jazykový konstrukt, který nabízí skládání tříd z "balíčků metod".
- Např. Pharo, Scala: `trait`; Ruby, JavaScript: `mixin`; Java (od verze 8) možnost implementované metody v `interface` (Default Method).

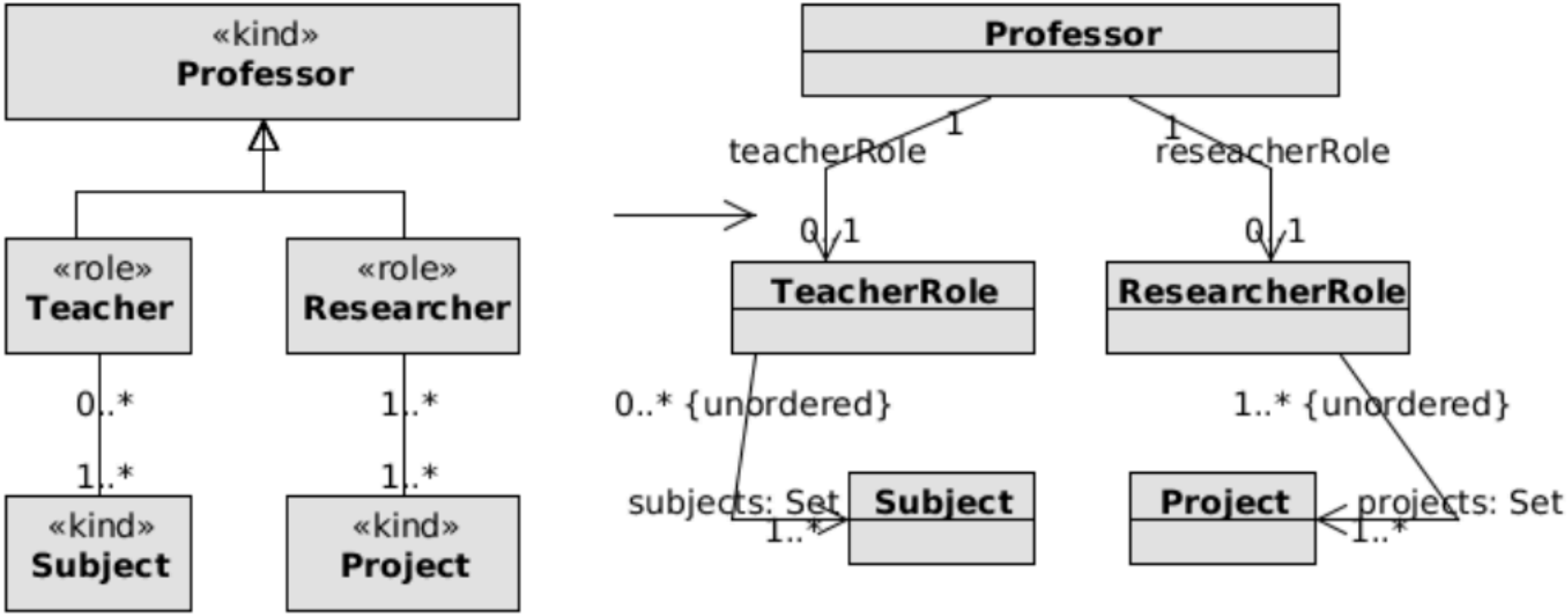
Implementace sortálních typů

- OntoUML nás vede k zamyšlení nad tím, co budeme chápat jako **identitu objektu**.
- Sortály pouze říkají: "Existuje jednoznačný princip identity" a tento je navíc čistě kognitivní
- Je tedy třeba zvolit vhodnou implementaci:
  - *Umělý identifikátor* -- typicky primární klíč v RDB, Object Identifier - OID, UUID, aj.
  - *Přirozený identifikátor* -- soubor identifikujících vlastností, např. implementace identity Osoba jako jméno + příjmení + datum narození

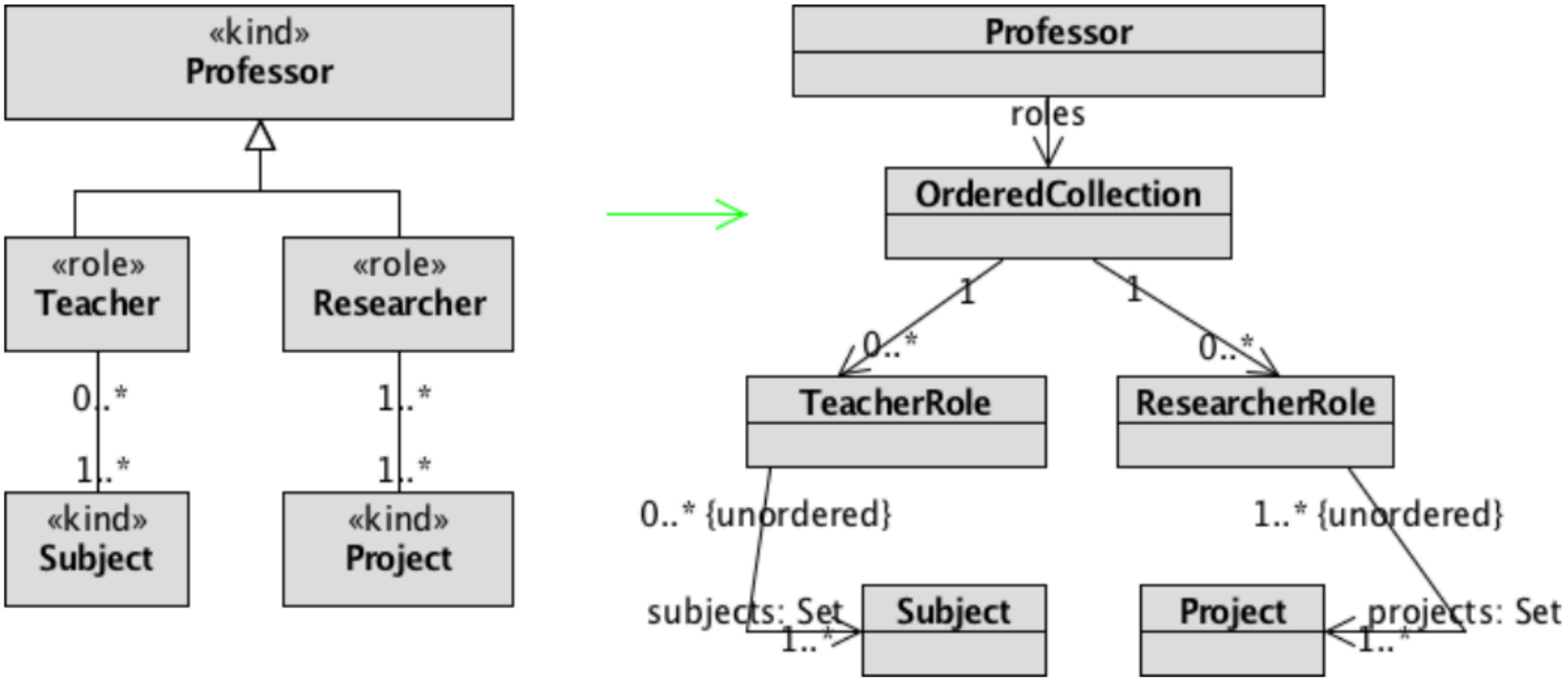
Implementace «Role»

- Každá entita typu «Role» bude opět implementována jako třída.
- Problém ale nastává s dědičností, protože dědění v OOP je **vždy rigidní** – jakmile vytvoříme instanci podtřídy, už ji nemůžeme zrušit, aniž bychom současně nezrušili i její část vycházející z nadtřídy.
- Jak tedy dosáhneme potřebné anti-rigidity, tj. možnosti přidávat a odebírat instanci podtřídy?
- Nahradíme dědění **skládáním**.

Implementace «Role» – varianta 1

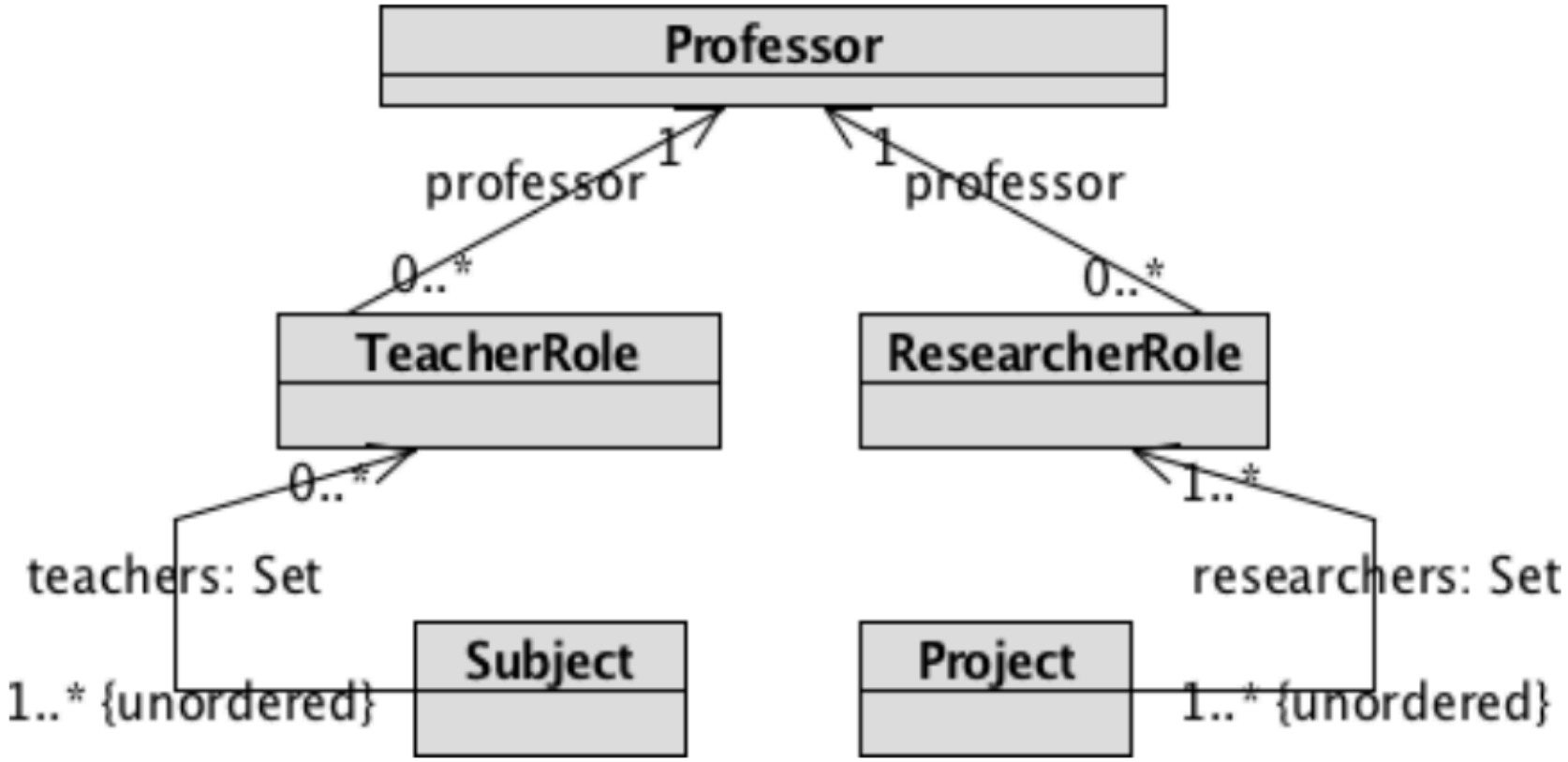


Implementace «Role» – varianta 2



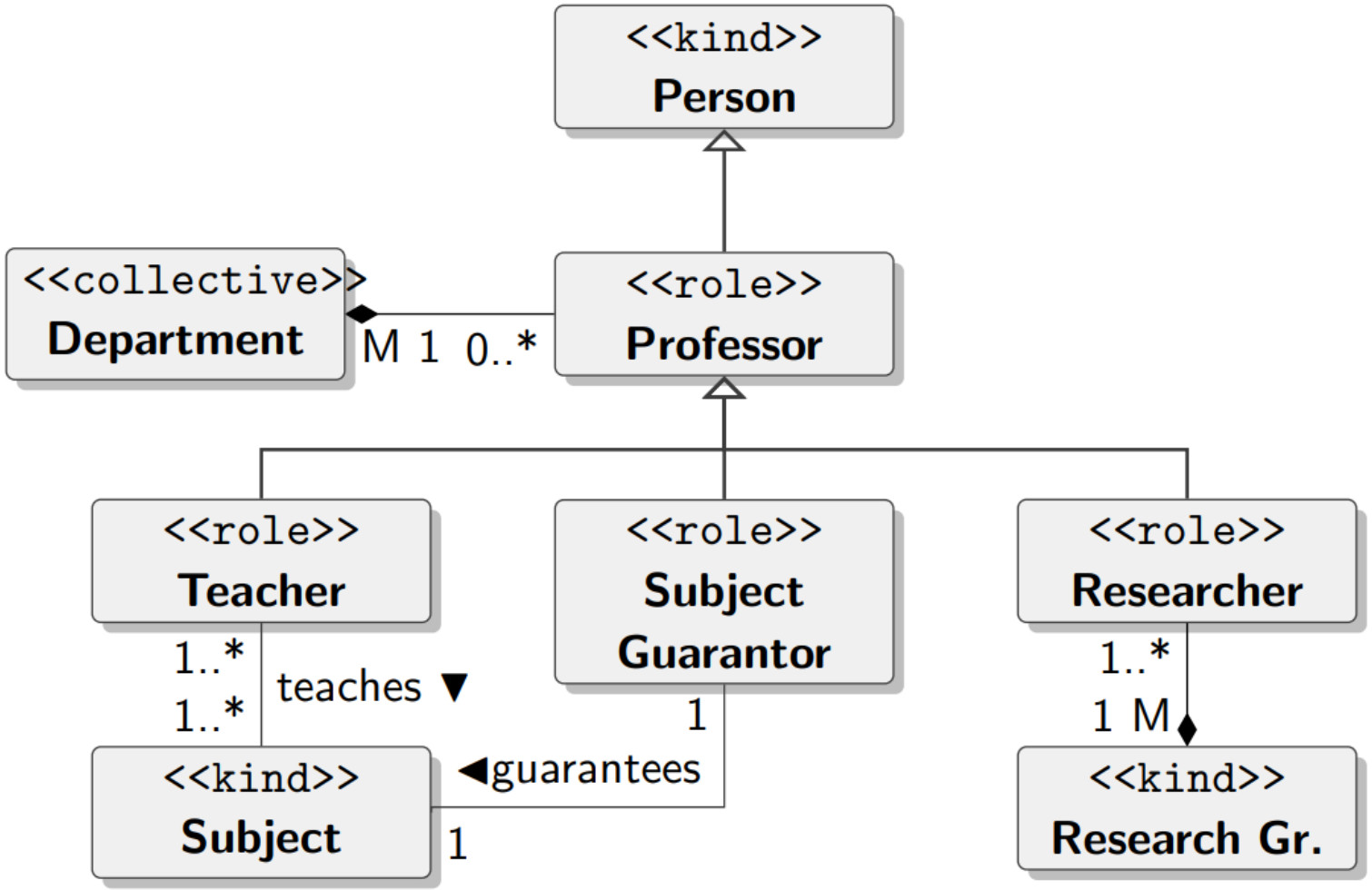
- Flexibilní práce s kolekcí rolí, ale složitější přístup k rolím.

Implementace «Role» – varianta 3

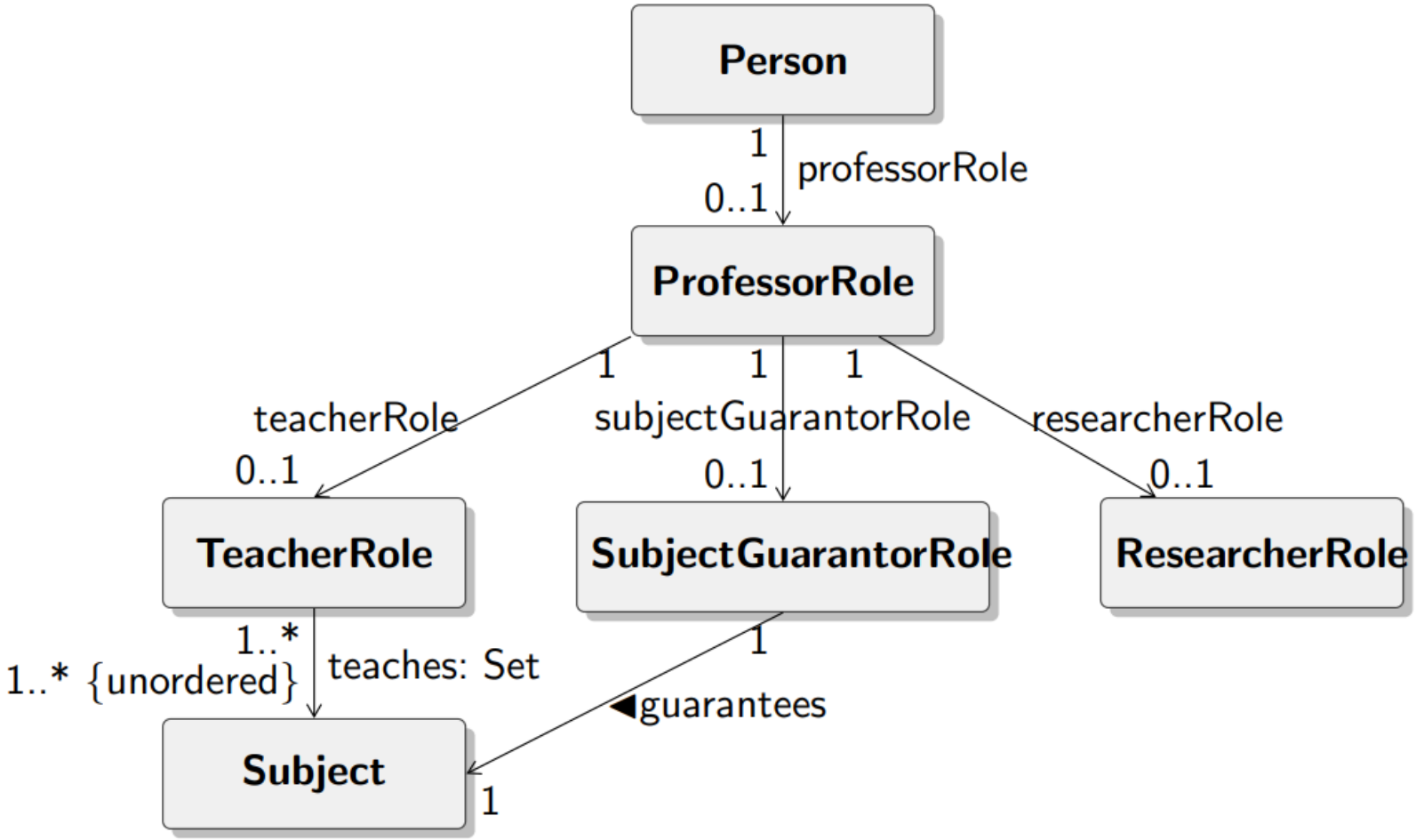


- Pro případ, kdy potřebujeme snadnou navigaci opačným směrem.

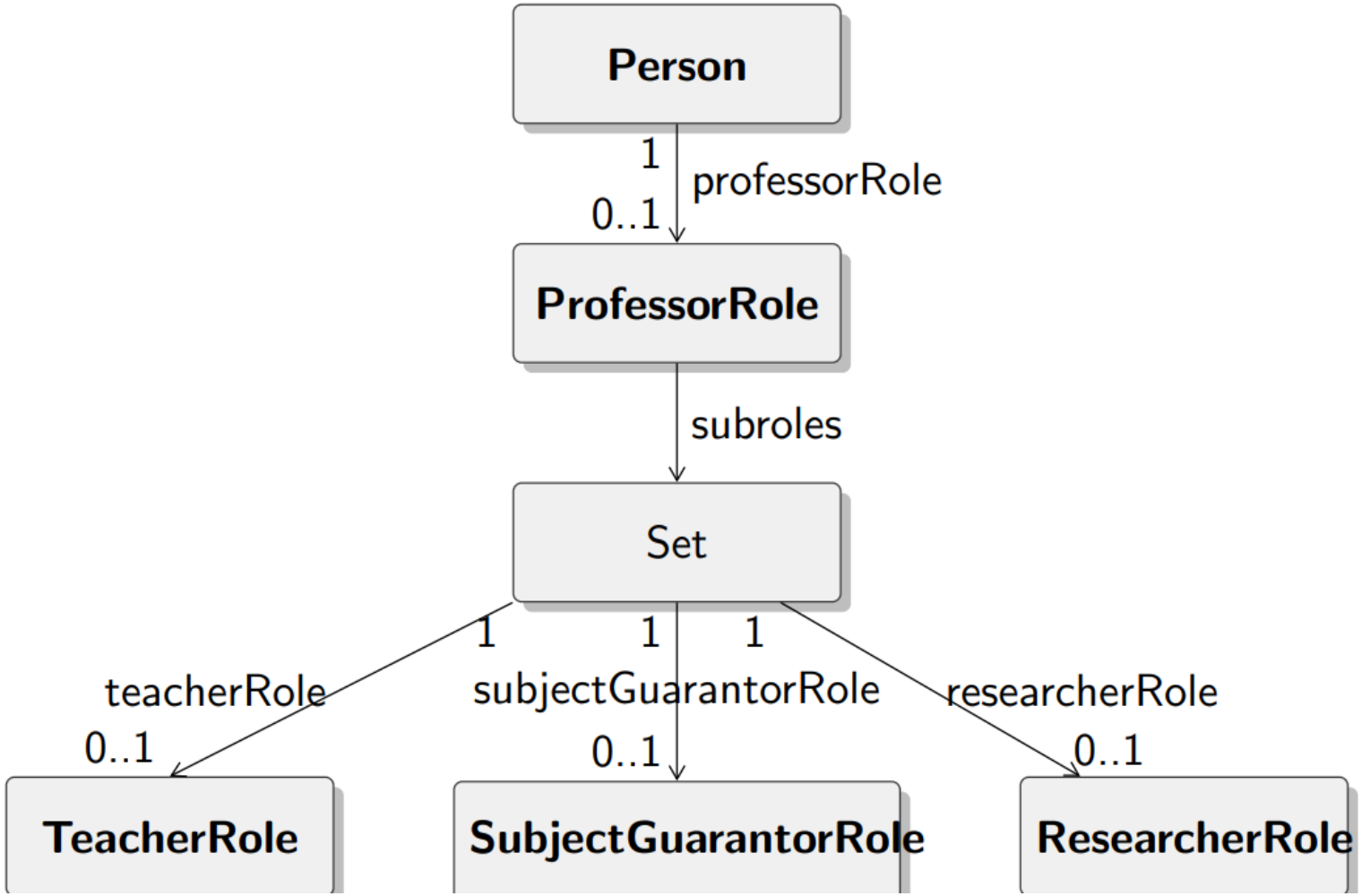
Implementace podrolí



Implementace podrolí – varianta 1

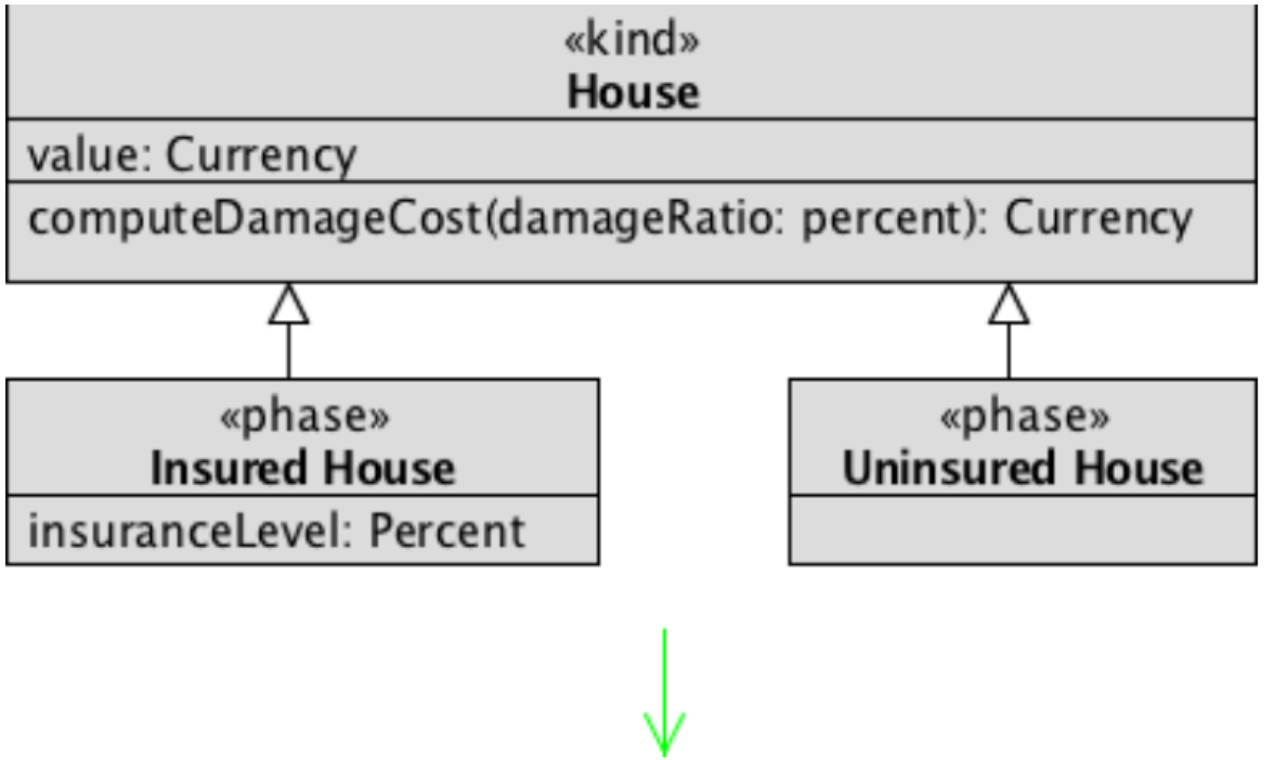


Implementace podrolí – varianta 2

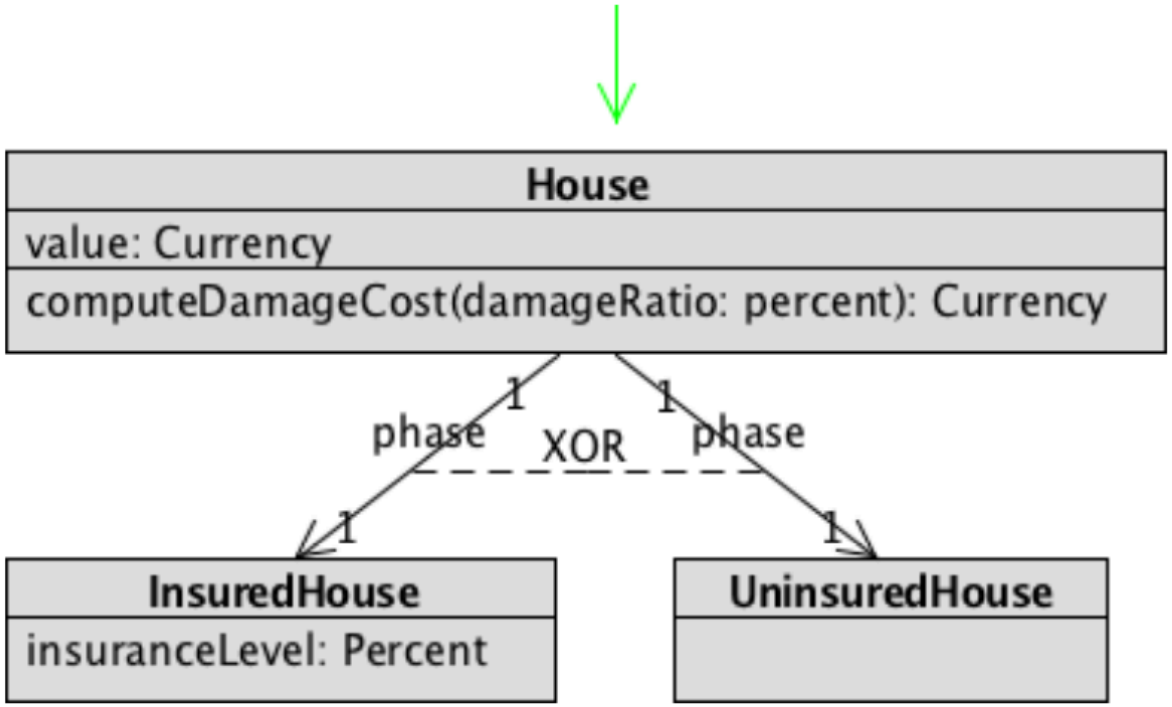


Implementace «Phase» – varianta 1

- Fázi implementujeme pomocí návrhového vzoru State (Stav).

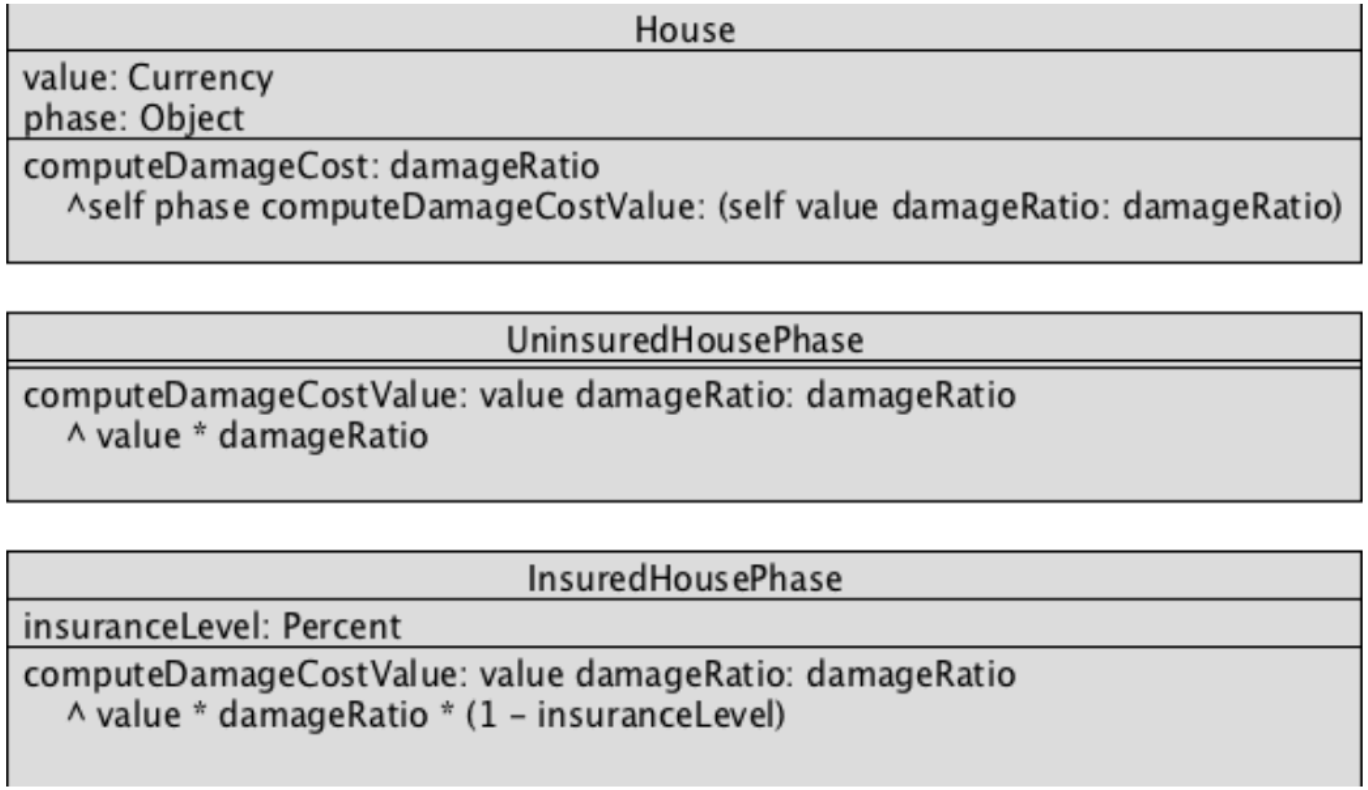


Implementace «Phase» – varianta 1



Implementace «Phase» – varianta 1

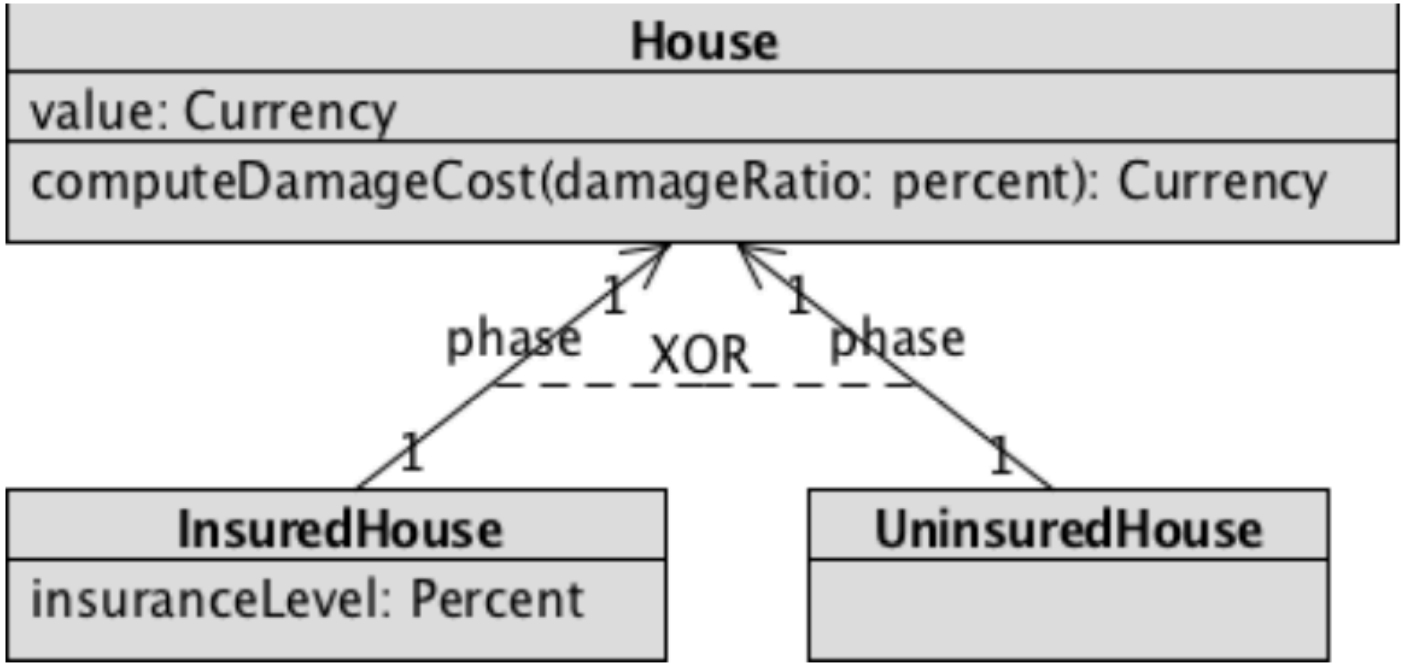
- Chování objektu na základě fáze je realizováno **delegováním**.



- Tento návrhový vzor je použitelný i pro disjunktní role.

Implementace «Phase» – varianta 2

- Podobně jako v případě «Role» můžeme potřebovat navigaci v opačném směru.

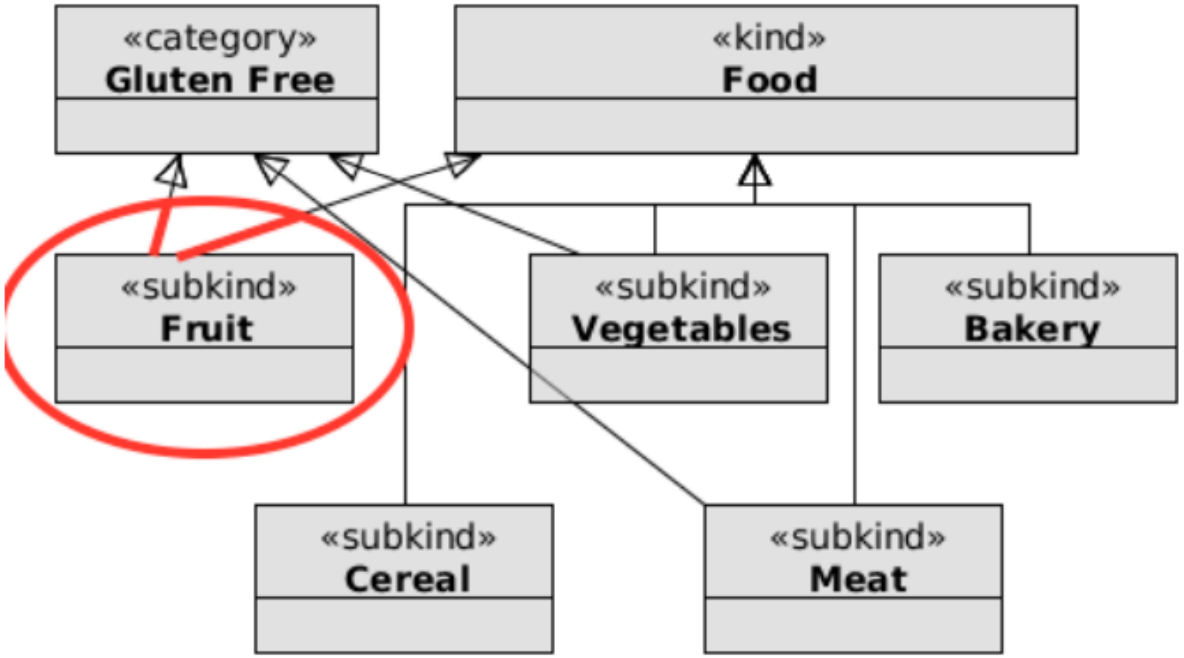


- Pravidlo **XOR** je (stejně jako obecně multiplicity na straně bez šipky) nutno ošetřit programově.

Implementace non-sortálních typů

- Jelikož non-sortály slouží jako další *dimenze kategorizace*, typicky budou vytvářet situace s **vícenásobnou dědičností**.

Implementace «Category»

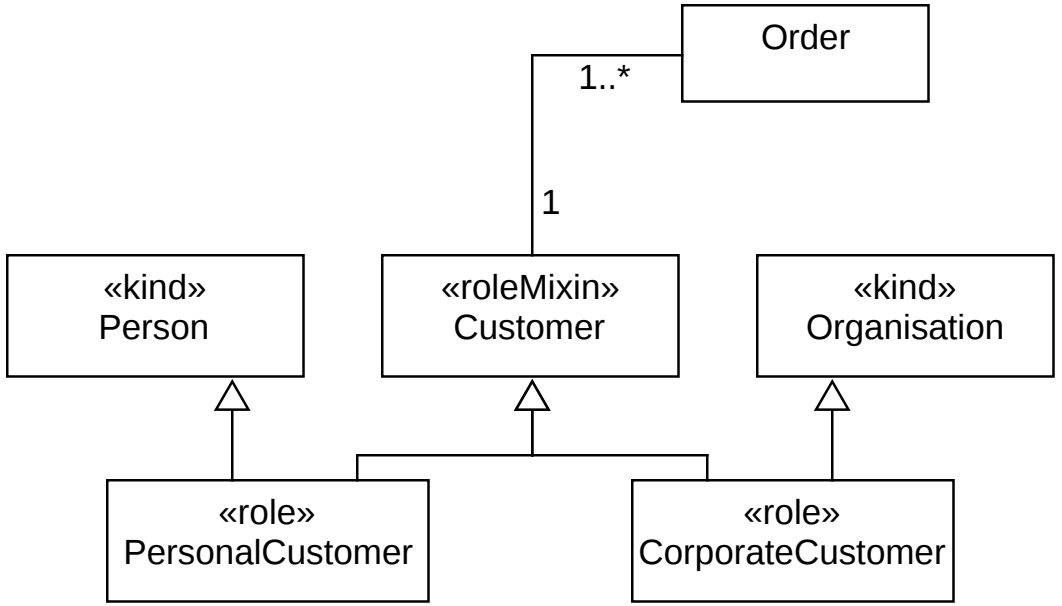


- U systémů, které nepodporují vícenásobnou dědičnost, je třeba dědičnost(i) nahradit skládáním (příp. trait/mixin).
- Jelikož potom budeme muset delegovat zprávy z celku na část, je vhodným vodítkem počet metod (zde `Food` vs `Gluten Free`)

Implementace «Category»

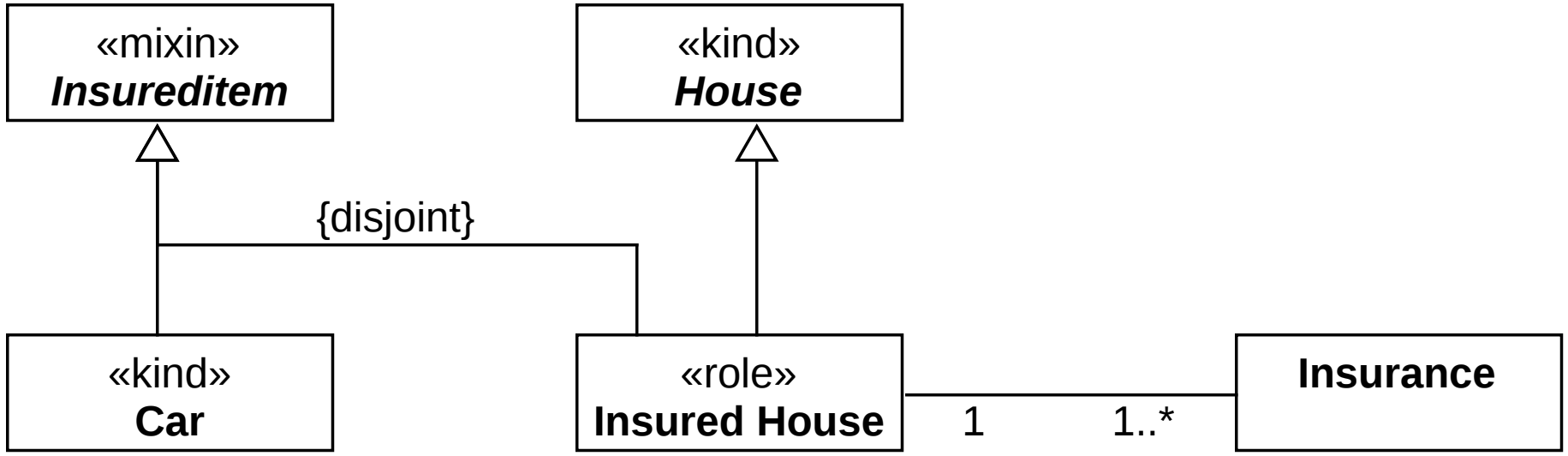
- Záleží i na tom, jak v implementaci budeme potřebovat realizovat `Gluten Free` -- pokud to bude např. jen indikace "Je to bezlepková potravina", můžeme implementovat jednoduše i jen např. jako metodu `isGlutenFree(): boolean` třídní metodu.
- Toto platí obecně i u všech "receptů" předchozích i následných -- při implementaci mohou "zbytečné" třídy být nahrazeny pouze metodou/atributem.

Implementace «RoleMixin»



- *Anti-rigidní* dědění `Person` -- `PersonalCustomer` se změní na skládání, čímž zanikne problém vícenásobné dědičnosti.

Implementace «Mixin»

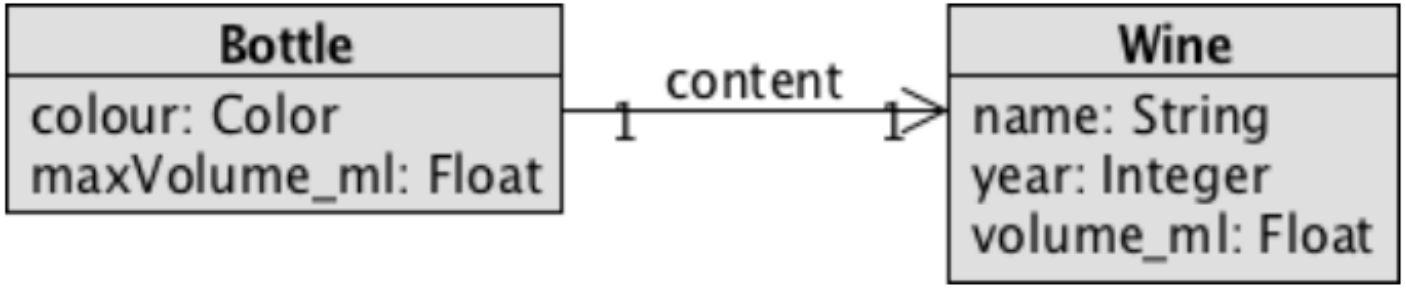


- *Anti-rigidní* dědění `House` - `InsuredHouse` se změní na skládání, čímž zanikne problém vícenásobné dědičnosti.

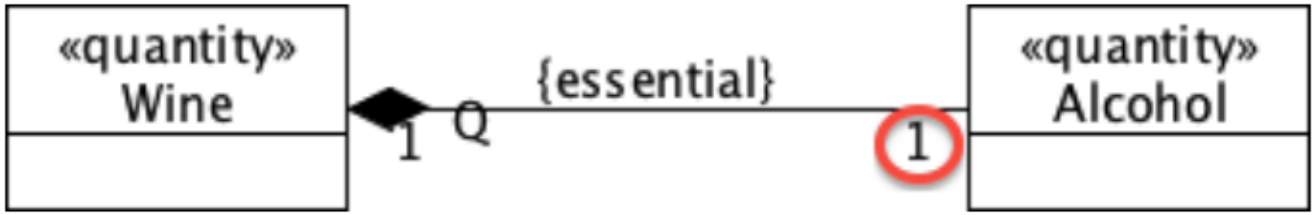
Implementace «Quantity» a «subQuantityOf»



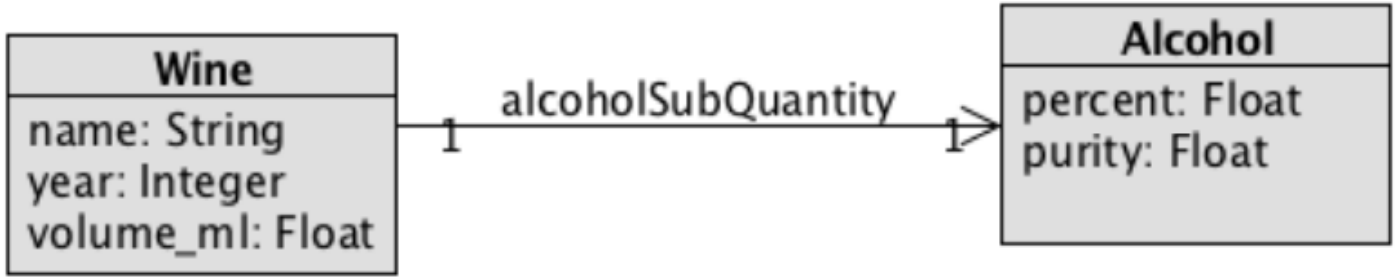
- «Quantity» je opět implementována jako standardní třída.
- Relaci «containment» implementujeme skládáním a nazýváme obvykle *content* (obsah).



Implementace «Quantity» a «subQuantityOf»



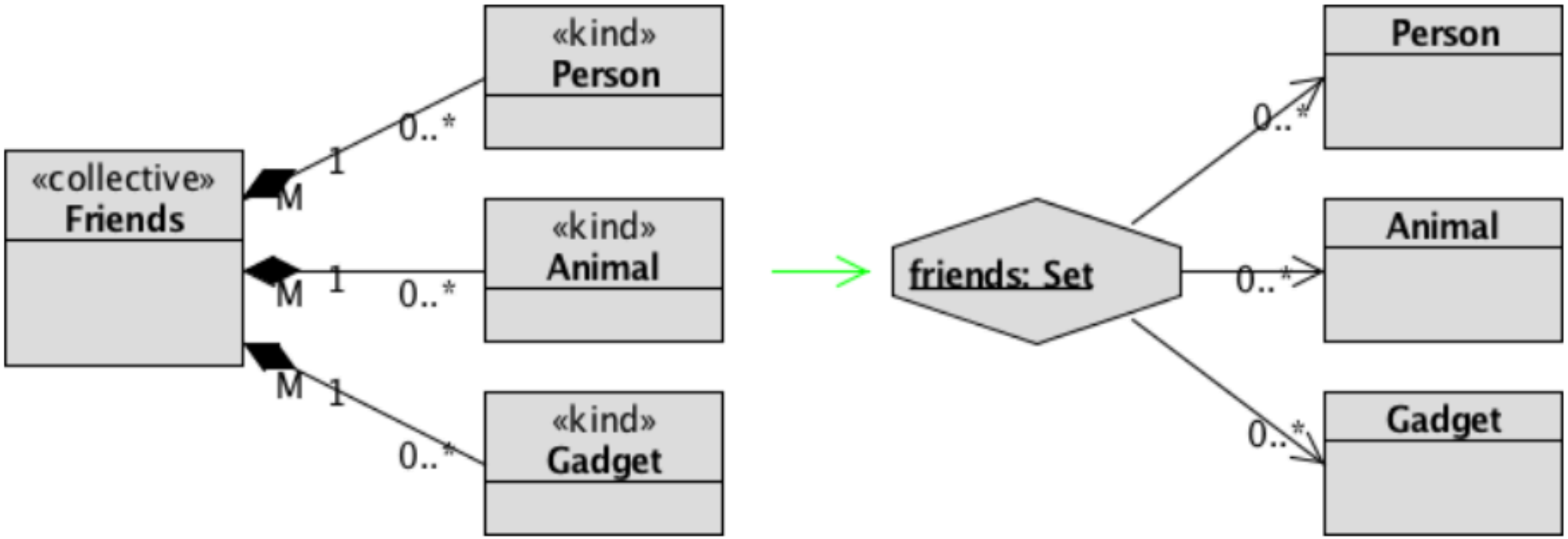
- «Quantity» navázaná vztahem «subQuantityOf» je analogicky třída.



Implementace «Collective»

- «Collective» představuje soubor rovnocenných prvků.
- Možnosti implementace:
  - Použijeme přímo nějakou variantu **kolekce** (collection) nabízenou v OOP knihovnách.
  - Tuto "obalíme" navíc do třídy -- pokud potřebujeme další atributy či metody.
- Prvky mohou být obecně **různých typů**.
- U dynamicky typovaných jazyků (Pharo, Python) toto není problém, u staticky typovaných jazyků (Java, C++, C#) je kombinace vícero typů prvků v jedné kolekci složitější a vyžaduje typicky implementaci *něčeho navíc* (interface nebo umělá nadtřída).

Implementace «Collective»

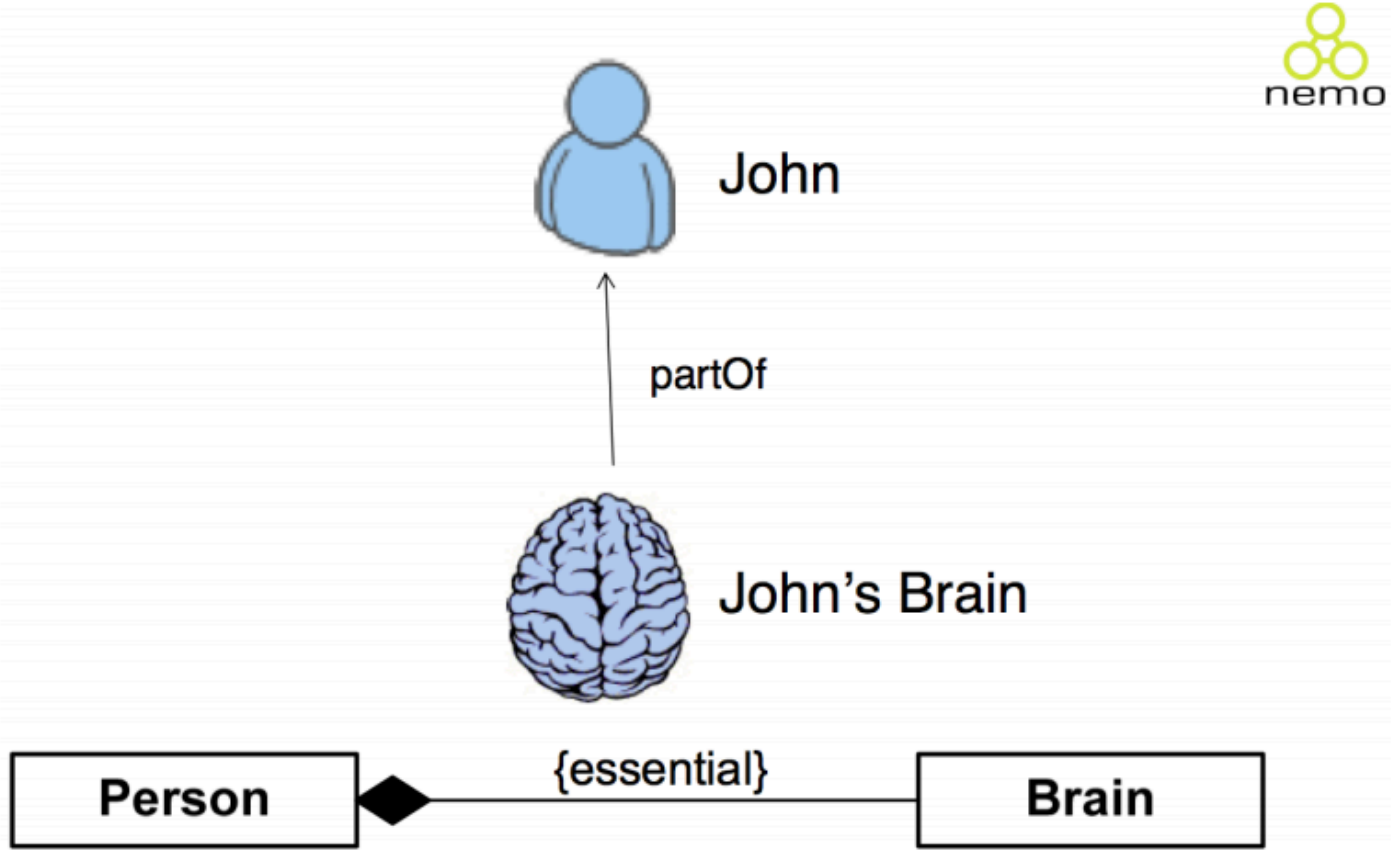


- Kolektiv se tedy transformuje na *instanci* kolekce, kterou uložíme do instanční proměnné.
- Lze říci, že soubor všech kolektivů z modelu představuje **objektovou databázi**.

Implementace funkčního celku

- **Dobrá zpráva**: funkční celek představuje skládání, tedy je implementován zcela přímočaře (s ošetřením multiplicit).

Implementace `{essential}`



Implementace `{essential}`

- V implementaci musíme zajistit, aby *instance* části nebyla vyměněna.
- Toto zajistíme takto:
    i. Vyrobit konstruktor Celku přijímající jako parametr esenciální Část.
    ii. Nevyrábět setter pro Část (tj. nepůjde zvenku změnit).
    iii. Dodržet, že v metodách uvnitř Celku nikde neprovedeme výměnu instance esenciální Části.

Implementace `{inseparable}`

- Implementace `{inseparable}` části není triviální, jelikož z Části nejsme schopni přímo kontrolovat chování Celku.
- Nemůžeme tedy jednoduše zajistit, aby jiný Celek si Část k sobě nesložil, ovšem můžeme zajistit, aby původní Celek svoji Část *nepustil* (podobně jako u `{essential}`).

Implementace `{immutablePart}` a `{immutableWhole}`

- Zachování Části anti-rigidního Celku (`{immutablePart}`) lze nejjednodušeji docílit tím, že Celek při zániku nebudeme mazat, ale pouze archivovat.
- Při použití v dalším světě tedy použijeme Celek rovnou s původní Částí.
- Pro `{immutableWhole}` platí analogicky to samé, co pro `{immutablePart}`.

Implementace Aspektů

- «Relator» -- přímočaré, opět jen je třeba řešit směry vazeb
- Interní Aspekty («Quality», «Mode») -- přímočaré + možnost optimalizace na atributy

Shrnutí transformace OntoUML do OO

- Čistý objektový model nabízí tyto konstrukty:
  - objekt (třída) s atributy a metodami,
  - skládání objektů,
  - dědění mezi třídami.
- Objektová implementace je tedy postavena na těchto konstruktech.
- V OOP je dědičnost vždy rigidní = nelze ji použít pro dědičnost anti-rigidních typů – musíme ji *emulovat* skládáním.
- Směr skládání je možné typicky zvolit – jedná se o stejnou problematiku jako u implementace asociací.
- U systémů s jednoduchou dědičností zajišťujeme implementaci vícenásobné dědičnosti opět skládáním či speciálními konstrukty (`trait`, `mixin`).

Shrnutí transformace OntoUML do OO

- Některá strukturální omezení (např. povinnost v multiplicitě) zajišťujeme vhodným ošetřením v kódu metod.
- Agregované typy («Quantity» and «Collective») lze implementovat jako kolekce nebo třídy v závislosti na tom, zda potřebujeme další atributy.
- Aspekty dle potřeby implementujeme jako atributy nebo třídy.
- Obecně platí, že uvedené "recepty" můžeme po úvaze zjednodušit.
- K zamyšlení:
  - Dědění (i `trait` a `mixin`) v OOP není nic jiného než skládání realizované runtime systémem *za vás*.

Časté otázky

„Proč nemůžu OntoUML jednoduše přepsat třídu za třídou do Javy?“

Některé konstrukce mají jinou sémantiku: člověk například může získávat a ztrácet role, zatímco běžný objekt v Javě za života svou třídu nemění. Proto přednáška převádí role pomocí skládání. Cílem transformace je zachovat význam a omezení.

Konec přednášky